SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

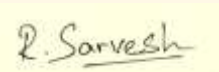
CO2 ASSESSMENT TOOL 2 – Architecture Design Assignment

Course Code:	CSA10 / CSA1021	**Course Title:**	Software Engineering
CO Assessed:	CO2	**Assessment:**	Assessment Tool 2 – Architecture Design Assignment
Weightage:	20%	**Total Marks:**	25
CO2 Statement:	*Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills*		
Student Name:	R SARVESH	**Reg. No.:**	192521143
Date:	30/10/2026	**Duration:**	60 minutes

Code of Conduct

I R SARVESH / 192521143 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student:



Annexure A – Architecture Design Assignment

Instructions to Students:

Each student will be assigned an **individual software project problem statement**. Based on the assigned problem, analyze the system requirements and design an appropriate software architecture by applying software engineering principles. Your submission should demonstrate your ability to select, justify, and evaluate a suitable architectural style.

Question

Based on your **assigned problem statement**, perform an **Architecture Design Analysis** and prepare an architecture design report by answering the following questions:

1. Analyze System Requirements

- Study the given problem statement and identify the system objectives.
- Analyze the functional and non-functional requirements that influence the software architecture.
- Identify key quality attributes such as scalability, maintainability, reliability, availability, performance, and security.

2. Select a Suitable Software Architecture

- Select an appropriate architectural style for the proposed system (e.g., Layered Architecture, Client-Server, MVC, Microservices, Event-Driven, Service-Oriented Architecture, Serverless, or Hybrid Architecture).
- Justify why the selected architecture is the most suitable for the given problem.

3. Draw the Software Architecture Diagram

- Design a clear architecture diagram illustrating the overall structure of the proposed system.
- Include all major layers/components, databases, external systems, APIs, and communication mechanisms.
- Use appropriate software architecture notations and labels.

4. Explain Components and Their Interactions

- Describe the purpose and responsibilities of each architectural component.
- Explain how the components communicate and exchange data.
- Illustrate the overall workflow of the system.

5. Discuss Quality Attributes
Evaluate how the proposed architecture addresses the following aspects:

- Scalability
- Security
- Performance
- Reliability
- Maintainability
- Availability

6. Justify Architectural Decisions

- Explain the rationale behind your architectural choices.
- Discuss the trade-offs considered while selecting the architecture.
- Explain how the chosen architecture supports future enhancements, system integration, and long-term maintainability.

Rubrics for Calculation

Evaluation Criteria	Excellent (4 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement Mark)
System Requirement Analysis (30%)	Thoroughly analyzes the problem statement, accurately identifies functional and non-functional requirements, and clearly explains quality attributes influencing the architecture.	Analyzes most requirements with minor omissions or limited explanation.	Identifies only basic requirements with partial analysis.	Requirement analysis is in inaccurate, or lacks clarity.
Selection of Software Architecture (25%)	Selects the most appropriate architectural style with strong technical justification aligned to system requirements.	Selects a suitable architecture with reasonable justification.	Architecture is partially suitable with limited justification.	Inappropriate architecture or justification is missing.
Architecture Diagram and Component Design (20%)	Produces a clear, complete, and well-structured architecture diagram with all major components, interfaces, and interactions accurately represented.	Diagram is mostly complete with minor omissions or notation issues.	Diagram includes only essential components and lacks sufficient detail.	Diagram is incomplete, un technically incorrect.
Component Interaction and Quality Attribute Analysis (15%)	Clearly explains component responsibilities, interactions, and thoroughly discusses scalability, security, performance, reliability, maintainability, and availability.	Explains most components and quality attributes with minor gaps.	Provides limited explanation of interactions and addresses only some quality attributes.	Component interactions ar attributes are poorly expla missing.
Architectural Justification and Technical Presentation	Provides strong justification for architectural decisions, discusses	Provides adequate justification with	Limited justification with minimal	Justification is weak or abs report lacks organization a

Criteria	**Mark**
System Requirement Analysis **(30%)**	/5
Selection of Software Architecture **(25%)**	/5
Architecture Diagram and Component Design **(20%)**	/5
Component Interaction and Quality Attribute Analysis **(15%)**	/5
Architectural Justification and Technical Presentation **(10%)**	/5
TOTAL	/25

1. Autonomous Drone-Carrier Mesh Network for Hyper-Local Medical Logistics

Industry Context

Modern municipal emergency response infrastructures are heavily bottlenecked by urban traffic congestion, grid design limits, and unpredictable weather anomalies. During critical medical crises—such as sudden cardiac arrests, acute anaphylaxis, or postpartum hemorrhages—the delivery time of life-saving medical supplies (e.g., Automated External Defibrillators (AEDs), EpiPens, rare blood types, or anti-venoms) determines patient survival. Traditional ambulance response times in dense metropolises often stretch beyond the critical 4-to-6-minute survival window.

While commercial drone delivery networks are emerging, individual drone units suffer from limited operational ranges, rapid battery depletion, and high vulnerability to sudden micro-climate shifts. Centralized cloud-based dispatch systems introduce single points of failure and high network latency penalties, which are unacceptable in life-or-death scenarios. There is an immediate structural demand for a decentralized, edge-computed logistics platform that treats automated delivery vehicles, municipal rooftops, and public transit nodes as a cohesive, self-healing transit mesh network.

Problem Statement

Design and develop a highly concurrent, fault-tolerant Autonomous Drone-Carrier Mesh Network that dynamically routes, schedules, and dispatches automated medical delivery drones across dynamic smart city topographies.

The platform must ingest high-velocity streaming telemetry—including real-time 3D drone coordinates, localized wind velocity vectors, active airspace geofences, battery health diagnostics, and regional cellular signal strengths—from hundreds of airborne units and municipal charging hubs concurrently.

Using decentralized routing engines, the platform must coordinate drone handovers between automated charging platforms mounted on moving public transit buses and static municipal smart buildings. The solution must run autonomous collision avoidance, handle sudden edge computing node dropouts, and guarantee zero-latency emergency rerouting without relying on continuous, active cloud infrastructure connectivity.

Objectives

The platform should:

- **Ingest and Process Sub-Second Telemetry:** Capture high-frequency 3D spatial positions and hardware diagnostic data from hundreds of active airborne drone arrays without packet loss.
- **Orchestrate Dynamic 4D Routing:** Compute optimal, collision-free 4D flight corridors (latitude, longitude, altitude, time) that adapt dynamically to shifting wind patterns and temporary no-fly zones.
- **Manage Moving Node Handovers:** Coordinate micro-second landing and deployment schedules onto mobile public transit carrier platforms using localized edge mesh communication loops.
- **Enforce Edge Autonomy & Self-Healing:** Implement decentralized routing protocols that allow the local swarm to self-heal and re-allocate tasks autonomously if the central cloud goes offline.
- **Ensure Mission-Critical Prioritization:** Design a strict traffic prioritization engine that automatically clears airspace and throttles commercial data streams when a high-priority medical alert is triggered.
- **Generate Cryptographic Provenance Audits:** Maintain unalterable, cryptographically signed operational logs to satisfy civil aviation authorities and medical privacy compliance boards.

Expected Outcomes

The proposed platform will:

- **Minimize Life-Saving Delivery Windows:** Reduce the average municipal transit window for critical medical payloads to under 4 minutes.
- **Extend Fleet Operational Reach:** Increase effective drone flight ranges by up to 300% through optimized, multi-stage mobile carrier handovers.
- **Guarantee High System Availability:** Achieve 99.999% network uptime for emergency dispatch tracking via fully decentralized edge computing worker networks.
- **Lower Operational Asset Losses:** Eliminate mid-air drone collisions and forced emergency landings via highly responsive, predictive collision avoidance networks.
- **Provide Regulatory Transparency:** Enable seamless compliance tracking for aviation authorities through instantly generated, immutable flight performance audit trails.

Annexure A – Architecture Design Analysis Report

Project Title: Autonomous Drone-Carrier Mesh Network for Hyper-Local Medical Logistics

1. Analyze System Requirements

1.1 System Objectives

The system is a decentralized, fault-tolerant medical logistics platform. It routes, schedules and dispatches autonomous delivery drones across a smart city. Drones, municipal rooftops, smart buildings and public-transit buses act together as a self-healing mesh. The architecture must support the following objectives:

1. Ingest and process sub-second telemetry (3D position, wind vectors, battery health, signal strength, geofence status) from hundreds of drones and charging hubs without avoidable packet loss.
2. Compute collision-free 4D routes (latitude, longitude, altitude, time) that adapt to wind changes and temporary no-fly zones.
3. Coordinate mission handovers between drones and moving or static carrier platforms.
4. Continue supported local operations, with self-healing task reallocation, when central cloud connectivity is lost.
5. Give critical medical missions strict priority over commercial workloads without violating airspace or collision-avoidance rules.
6. Maintain signed, tamper-evident audit records for aviation authorities and privacy compliance boards.

The stated targets are a delivery window under 4 minutes, up to 300% extended range and 99.999% availability for emergency dispatch tracking. These are **design targets to be verified through simulation, benchmarking and controlled trials**, not guaranteed outcomes. "Zero-latency" rerouting is interpreted as local decision-making with no cloud round trip. Physical flight time, radio delay and weather remain outside software control.

1.2 Stakeholders Influencing the Architecture

Stakeholder	**Architectural Concern**
Emergency medical teams / operators	Fast, reliable request registration and mission visibility
Drone operators	Real-time status, manual intervention capability
Municipal and aviation authorities	Geofence enforcement, auditable compliance
Bus, rooftop and building platform operators	Standard, safe handover and charging

	interfaces
Security and compliance team	Authentication, encryption, key management, audit integrity
Development and operations teams	Modular services, documented APIs, testability, deployability

1.3 Functional Requirements

ID	**Functional Requirement**
FR1	Register and validate emergency medical delivery requests, creating a unique mission ID and status.
FR2	Ingest, validate, timestamp and store telemetry from drones, buses and charging hubs.
FR3	Display the latest accepted drone, platform and mission status to operators.
FR4	Compute 4D routes and recalculate them when wind, no-fly zones or platform availability change.
FR5	Validate every proposed route against geofences and safety constraints before authorization.
FR6	Run decentralized, onboard-assisted collision avoidance among drones in shared airspace.
FR7	Coordinate drone-to-platform handovers, with confirmation and recoverable failure handling.
FR8	Schedule missions by priority, giving eligible medical emergencies precedence over commercial tasks.
FR9	Throttle non-critical data streams when a high-priority alert is active.
FR10	Detect edge-node dropouts and reassign eligible tasks to healthy nodes.
FR11	Continue supported local operations during cloud loss and reconcile state on reconnection.
FR12	Generate signed, tamper-evident audit records and support integrity verification.

1.4 Non-Functional Requirements and Quality Attributes

Quality Attribute	**Requirement**	**Architectural Influence**
Performance	Telemetry ingestion and local routing decisions within a sub-second budget, verified by benchmarking	Edge processing, asynchronous event streaming, in-memory state
Scalability	Hundreds of concurrent drones and many edge nodes	Horizontally scalable stateless services, partitioned event streams
Availability	99.999% target for emergency dispatch tracking	Redundant edge nodes, no mandatory cloud dependency
Reliability	No silent loss of missions or telemetry	Durable local logs, idempotent processing, recovery workflows
Safety	Emergency priority must never override airspace or collision rules	Safety invariants enforced in a dedicated validation layer
Security	Authenticated drones, nodes and users; encrypted data; signed records	Mutual TLS, role-based access, hardware-backed key storage
Consistency	Conflict-free mission ownership during handover	Consensus for safety-critical state, eventual consistency for non-critical data
Fault tolerance	Survive node dropouts, stale feeds and partitions	Health monitoring, leader election, fail-safe modes
Maintainability	Independently testable and deployable components	Microservices, API-first contracts, containerization
Auditability	Verifiable mission history	Append-only signed log with hash chaining

1.5 Requirement Prioritization (MoSCoW)

Category	**Requirements**
Must Have	Emergency request registration (FR1), telemetry ingestion (FR2), geofence-validated routing (FR4, FR5), collision avoidance (FR6), emergency prioritization (FR8), edge autonomy during cloud loss (FR11), signed audit records (FR12),

	authentication and encryption
Should Have	Platform handover coordination (FR7), node dropout recovery (FR10), data-stream throttling (FR9), operator dashboard (FR3)
Could Have	Predictive wind modelling, battery-aware multi-hop route optimization, analytics dashboards
Won't Have	Full city-wide production deployment, final aviation certification, unrestricted autonomous operation

Kano classification:

- **Basic (Must-Be):** collision avoidance, geofence compliance and secure authentication. Their absence causes severe dissatisfaction, and their presence is simply expected.
- **Performance (Linear):** delivery time, telemetry latency and uptime. Satisfaction rises with performance.
- **Delighter:** extended range through multi-stage carrier handovers and predictive rerouting.
- **Indifferent:** cosmetic dashboard themes, which are excluded to save effort.

Requirements are elicited through focus groups with emergency teams, operators and aviation representatives. Prototypes of dispatch and monitoring screens validate ambiguous workflows. Requirement conflicts, such as speed against safety, are resolved through documented DACI decisions recorded as Architecture Decision Records.

2. Select a Suitable Software Architecture

2.1 Selected Architecture: Hybrid Edge-Native Event-Driven Microservices Architecture

The proposed system combines four patterns:

- **Microservices:** independently deployable services, each owning one capability and its own data.
- **Event-driven architecture:** services communicate asynchronously through event streams that represent state changes such as telemetry received, route invalidated, handover requested or priority alert raised.
- **Distributed edge computing:** services run on edge nodes (rooftops, smart buildings, buses) so critical decisions are made locally.
- **Serverless cloud tier:** non-time-critical, bursty workloads such as audit report generation, analytics and archival run as managed functions.

Services are containerized and orchestrated with a lightweight Kubernetes distribution on edge nodes. All interfaces are defined API-first with OpenAPI.

2.2 Why This Architecture Is Suitable

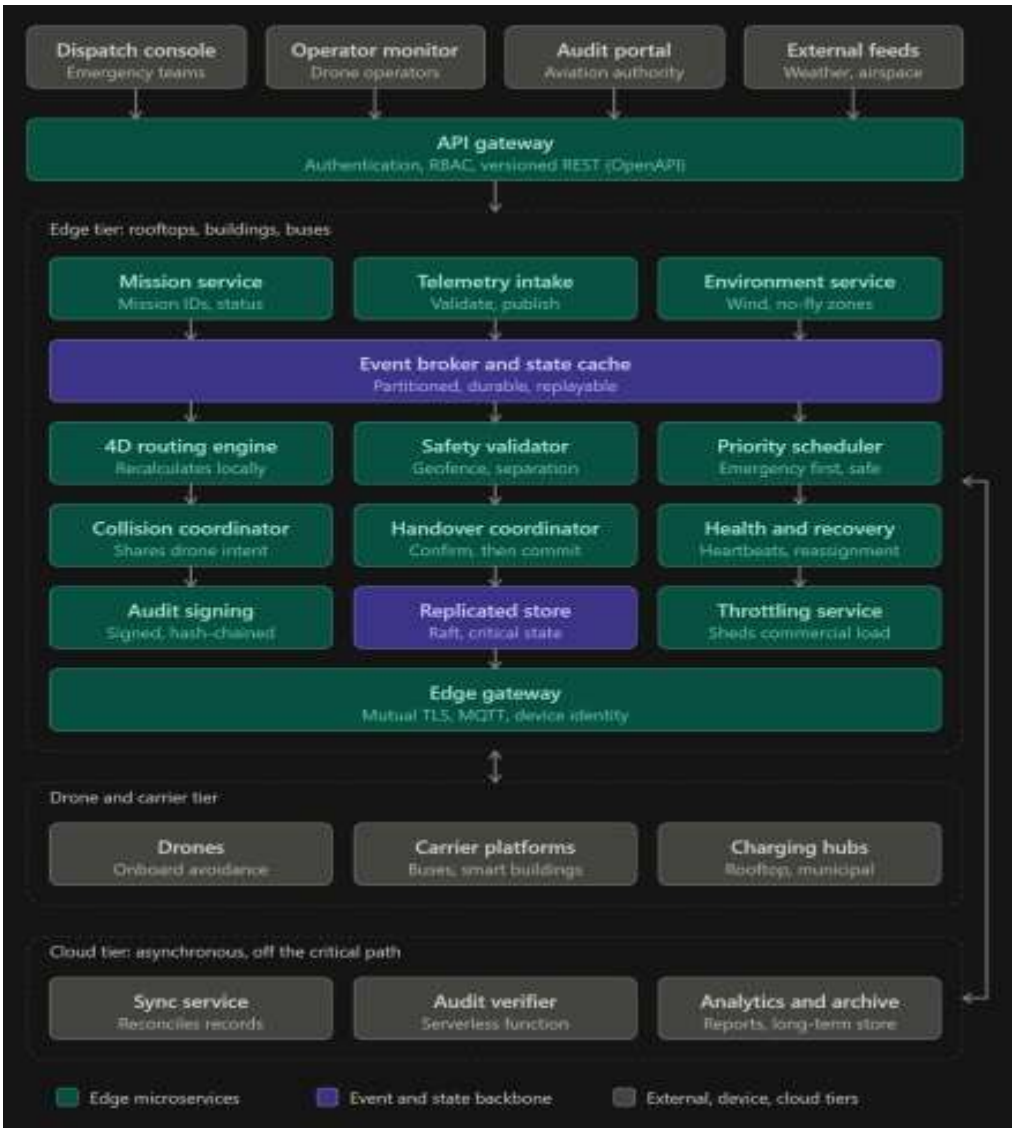
- **Local autonomy:** edge placement removes the cloud round trip from emergency rerouting and collision avoidance and removes the cloud as a single point of failure.
- **High-velocity telemetry:** an event stream absorbs bursts and lets consumers (routing, monitoring, audit) process the same data independently.
- **Independent scaling:** telemetry ingestion can scale separately from routing or auditing as drone counts grow.
- **Fault isolation:** failure of the dashboard or audit archiver does not stop routing or dispatch.
- **Self-healing:** health events trigger task reassignment among edge nodes.
- **Moving-platform support:** loosely coupled handover services handle buses that move in and out of coverage.
- **Extensibility:** new drone types, routing models or platform operators plug in through versioned APIs and event schemas.
- **Cost efficiency:** serverless functions are billed per use and suit intermittent compliance workloads.

2.3 Architecture Comparison

Architecture	**Advantages**	**Limitations for This Project**
Layered	Clear separation of concerns	Synchronous layer-to-layer calls add latency; tends toward centralization
Client–Server	Simple request/response	Central server is a single point of failure; poor fit for streaming telemetry
MVC	Good for operator interfaces	Covers only the presentation tier, not a distributed system
Pure Cloud Microservices	Elastic scaling	Depends on connectivity; unacceptable latency for life-critical decisions
Pure Serverless	Automatic scaling, low operations	Cold starts and vendor lock-in; unsuitable for continuous real-time control
Event-Driven only	Responsive, decoupled	Needs service boundaries and deployment structure to be maintainable
Hybrid Edge-Native Event-Driven Microservices	Local autonomy, scalable, fault-isolated, extensible	Higher operational complexity and consistency management (addressed in Section 6)

Selected solution: the hybrid architecture is the only option that satisfies local autonomy, real-time telemetry handling, independent scaling and fault-tolerant self-healing together.

3. Draw the Software Architecture Diagram



4. Explain Components and Their Interactions

4.1 Architectural Tiers

1. **Operator and External Tier:** emergency dispatch console, drone operator monitoring interface, aviation authority audit portal, and external feeds (weather, airspace and geofence authorities, transit schedules).

2. **Cloud Tier (non-critical path):** global coordination, long-term storage, analytics, audit archive and report generation. It is not required for local emergency operation.

3. **Edge Tier:** microservices on rooftop, building and bus nodes that perform real-time ingestion, routing, scheduling, handover and local audit signing.

4. **Drone and Carrier Tier:** drone onboard flight controllers (with local collision avoidance), carrier-platform controllers and charging hubs.

4.2 Component Responsibilities

Component	**Responsibility**
API Gateway	Authenticates requests, enforces role-based access, rate limits, routes to services; exposes versioned REST APIs (e.g., /api/v1/missions)
Mission Service	Validates requests, creates unique mission IDs, stores and updates mission status (US-01)
Edge Gateway	Terminates mutually authenticated connections from drones and carriers; forwards messages to the event stream
Telemetry Ingestion Service	Validates schema, timestamps and freshness of telemetry; rejects or quarantines malformed messages; publishes accepted events (US-02)
Event Stream / Local Message Broker	Distributes events by topic and partition; durable local buffering and controlled replay
State Cache	In-memory view of the latest drone, platform, wind and geofence state
Environment and Geofence Service	Maintains wind data, active no-fly zones and airspace restrictions
4D Routing Engine	Computes and recalculates routes using position, time, wind, battery and restrictions (US-03)
Route Safety Validator	Checks proposed routes against geofences and separation constraints; rejects unsafe routes
Collision Avoidance Coordinator	Shares intent between nearby drones and edge nodes; works with onboard avoidance so each drone retains a safe local fallback
Priority Scheduler	Orders missions and allocates resources; emergency tasks take precedence while safety rules stay mandatory (US-06)
Traffic Throttling Service	Reduces commercial data and task load when a critical alert is active
Handover Coordinator	Verifies platform availability and compatibility, runs the commit protocol and updates mission ownership atomically (US-04)
Cluster Health and Recovery	Detects node dropouts through heartbeats, elects leaders, reassigns tasks

Manager	and triggers recovery (US-05)
Local Replicated Store	Raft-replicated store for safety-critical state (mission ownership, airspace reservations); append-only local event log
Sync Service	Reconciles edge and cloud records after connectivity returns, flagging conflicts instead of overwriting
Audit Signing Service	Signs mission events with device-protected keys, chains hashes, and stores records in an append-only log (US-07)
Key and Identity Management	Issues and rotates certificates and keys; authenticates drones, nodes and users
Audit Verification Function (serverless)	Verifies signatures and integrity on demand for authorized verifiers; generates compliance reports
Analytics and Archive (cloud)	Long-term storage, performance analysis and reporting
Monitoring and Alerting	Centralized logs, metrics and traces; operator alerts

4.3 Communication Mechanisms

Path	**Mechanism**	**Reason**
Operator ↔ API Gateway	HTTPS / REST (OpenAPI)	Standard, developer-friendly, documented
Drone/carrier ↔ Edge Gateway	Lightweight publish/subscribe messaging (e.g., MQTT) over mutually authenticated TLS	Low overhead for constrained devices and unreliable links
Service ↔ Service (edge)	Asynchronous events through the broker; gRPC for latency-sensitive internal calls	Decoupling plus performance
Edge node ↔ edge node	Mesh messaging and replicated consensus for critical state	Decentralized coordination
Edge ↔ Cloud	Asynchronous store-and-forward sync	Tolerates disconnection
Cloud ↔ external feeds	Authenticated APIs and webhooks	Weather and airspace updates

4.4 Overall Workflow

1. **Request:** an emergency operator submits a medical delivery request. The gateway authenticates it, and the Mission Service validates it, creates a unique mission ID and publishes a MissionCreated event.
2. **Scheduling:** the Priority Scheduler assigns the mission to a suitable drone and, if needed, a carrier platform, applying emergency priority and signalling the Throttling Service.
3. **Telemetry:** drones and carriers continuously send telemetry to the Edge Gateway. The Ingestion Service validates it and publishes events that update the State Cache and monitoring views.
4. **Routing:** the Routing Engine builds a 4D route. The Route Safety Validator checks geofences and separation, and only validated routes are authorized.
5. **Execution and avoidance:** the drone follows the route. The Collision Avoidance Coordinator and onboard logic maintain separation.
6. **Dynamic change:** a wind change, new no-fly zone or battery warning publishes an event. The Routing Engine recalculates locally without waiting for the cloud.
7. **Handover:** when range or battery requires it, the Handover Coordinator verifies the receiving platform, executes a confirm-then-commit transfer and updates mission ownership consistently. If confirmation fails, the mission stays recoverable and the failure is logged.
8. **Failure recovery:** if an edge node drops out, the Health Manager detects the missing heartbeats, elects a replacement and reassigns eligible tasks. If the task cannot be verified as safe, it does not proceed merely to maintain availability.
9. **Audit:** each significant event is signed and appended to the audit log.
10. **Synchronization:** when the cloud connection is available, the Sync Service reconciles records, flags conflicts, and archives data. Authorities verify records through the audit portal.

5. Discuss Quality Attributes

5.1 Scalability

Stateless services such as ingestion, routing workers and scheduling can be replicated horizontally by the orchestrator. Telemetry streams are partitioned by drone ID or geographic zone, so consumers scale in parallel. Each city zone can be served by its own cluster of edge nodes, which allows growth without a central bottleneck. Cloud analytics and audit functions scale automatically on demand.

5.2 Security

- Mutual TLS between drones, nodes and services, with certificate-based device identity.
- Role-based access control for operators, supervisors, authorities and verifiers.
- Encryption in transit and, for patient and delivery data, at rest.
- Private keys held in hardware-backed or protected storage, with rotation and revocation.
- Message signing and replay protection to resist forged commands.
- Hash-chained signed audit records to detect tampering.
- Data minimization: only mission-necessary personal data is collected and access is role-restricted.

Tamper-evidence proves a record was not altered after signing. It does not prove the original data was truthful, so source authentication and validation are also enforced.

5.3 Performance

Processing at the edge removes the cloud round trip from critical decisions. The in-memory State Cache avoids repeated database reads. Asynchronous events prevent blocking. Lightweight messaging protocols reduce overhead on drone links. Load shedding and throttling protect critical flows during bursts. Telemetry latency, routing computation time, handover time and end-to-end dispatch time are measured separately through benchmarking and simulation rather than assumed.

5.4 Reliability

- Durable local event logs and controlled replay prevent silent data loss.
- Idempotent event handling avoids duplicate dispatch or double handover.
- Timeouts, bounded retries and circuit breakers contain failures.
- Telemetry freshness checks reject stale data.
- A two-phase confirm-then-commit handover keeps mission ownership unambiguous.
- Fail-safe behaviour: when safe execution cannot be verified, the system holds, returns or lands the drone at a designated safe location. It never continues merely to stay available.

5.5 Maintainability

Each microservice has a single responsibility and its own data store. Services are containerized for consistent development, test and production environments. API-first design means OpenAPI contracts are written before implementation. Mock servers let teams work in parallel, and contract tests and generated client SDKs reduce documentation drift. URL-path versioning (e.g., /api/v1/) with backward-compatible changes within a major

version allows safe evolution. Structured logging, distributed tracing, CI pipelines and automated tests support maintenance.

5.6 Availability

Multiple edge nodes cover each zone, so overlapping coverage survives single-node failure. Raft consensus for critical state tolerates minority node loss. Orchestrator self-healing restarts failed containers. Local operation continues during cloud outages. The cloud is deliberately kept off the critical path so that the 99.999% target applies to the decentralized emergency tracking function. Network partitions are handled by the following consistency policy:

Data Type	**Consistency Strategy**
Mission ownership, airspace reservations	Strong consistency (consensus); operations pause or fail safe if quorum is lost
Telemetry, monitoring views	Eventual consistency
Audit records	Append-only, merged by hash chain and timestamp on reconnection

This follows the distributed-systems trade-off between consistency and availability. Strong consistency is used where safety requires it and availability is favoured elsewhere.

5.7 Correctness and Safety Assurance (Design by Contract and Formal Verification)

Critical components specify explicit contracts:

Component	**Precondition**	**Post condition**	**Invariant**
Route Safety Validator	Route is complete and environment data is fresh	Returned route is geofence-compliant and separation-safe, or is rejected	No authorized route intersects an active no-fly zone
Handover Coordinator	Receiving platform has confirmed availability and compatibility	Exactly one entity owns the mission after commit	A mission never has two simultaneous owners
Priority Scheduler	Mission has a valid priority and resource request	Emergency tasks are ordered before eligible commercial tasks	Priority never bypasses airspace or collision constraints

Audit Signing Service	Event has a mission reference and timestamp	Signed record is appended to the chain	Records are never modified in place

Contracts are checked at runtime through assertions and contract tests. The handover commit protocol, leader election and priority-safety properties are suitable for model checking, using a tool such as TLA+, to explore all reachable states and show that unsafe states such as dual ownership or priority-induced airspace violation cannot occur. Formal verification is applied selectively because of its cost.

6. Justify Architectural Decisions

6.1 Rationale for Major Decisions

Decision	**Justification**
Edge-based processing	Removes cloud dependency and round-trip latency from life-critical decisions
Event-driven communication	Handles high-velocity telemetry and decouples producers from consumers; events queue safely if a consumer fails
Microservices	Independent deployment, scaling and fault isolation
Containers with orchestration	Consistent environments, automated scheduling, self-healing and horizontal scaling
API-first with OpenAPI	Stable contracts, parallel development, living documentation, generated SDKs and contract tests
Separate routing, validation and scheduling	Safety checks cannot be bypassed by optimization logic
Consensus store for critical state	Prevents conflicting mission ownership and airspace reservations
Onboard collision avoidance plus coordinated avoidance	Drones remain safe even if the edge link fails
Hash-chained signed audit log	Meets regulatory transparency and tamper-evidence needs
Serverless cloud functions	Cost-efficient for intermittent audit and analytics workloads
Design by Contract and selective formal verification	Raises confidence in safety-critical logic beyond what testing alone provides

6.2 Architectural Trade-offs

Benefit	**Trade-off**	**Mitigation**
Local autonomy	Distributed state is harder to keep consistent	Consensus for critical data, eventual consistency elsewhere, reconciliation on reconnect
Asynchronous events	Ordering, replay and duplicate handling complexity	Sequence numbers, idempotent handlers, deterministic replay
Microservices	Operational overhead and network latency between services	Container orchestration, co-location of latency-sensitive services, gRPC
Edge distribution	Limited compute and larger attack surface	Lightweight services, hardened nodes, mutual authentication, remote attestation
Automated rerouting	Incorrect data could cause unsafe routes	Data validation, safety validator, fail-safe modes, human override
Serverless	Cold starts and vendor lock-in	Used only for non-real-time workloads; portable interfaces
Formal verification	High expertise and effort	Applied only to handover, election and priority-safety properties

6.3 Support for Future Enhancements, Integration and Long-Term Maintainability

- **Advanced routing and prediction:** machine-learning wind prediction or swarm optimization can be added as new subscribers to existing event streams.
- **New platforms and operators:** additional bus fleets, buildings or charging providers integrate through standard handover APIs.
- **New drone types:** device adapters at the Edge Gateway isolate hardware differences from core logic.
- **Regulatory evolution:** audit and policy services can be updated independently as aviation rules change.
- **Integration with emergency systems:** hospital, ambulance and public-safety systems connect through versioned APIs and webhooks.
- **Safe evolution:** backward-compatible API versioning, schema evolution and containerized rolling updates avoid service disruption.
- **Phased delivery:** the architecture allows incremental sprints, with dispatch and telemetry first, then priority scheduling, routing, edge autonomy, handover, auditing and validation, each independently testable.

7. Conclusion

The Hybrid Edge-Native Event-Driven Microservices Architecture is well suited to the Autonomous Drone-Carrier Mesh Network. It places critical decision-making at the edge, uses event streams to absorb high-velocity telemetry, and separates routing, safety validation, scheduling, handover and auditing into independently scalable and testable services. API-first contracts, containerized deployment, consensus-backed critical state and signed audit records support scalability, availability, security and maintainability together.

The central design principle is that **safety takes precedence over speed and availability**. Emergency priority never overrides airspace or collision-avoidance rules, and the system fails safe when safe execution cannot be verified. The targets of under 4-minute delivery, up to 300% extended range and 99.999% availability remain design goals. They must be demonstrated through benchmarking, simulation, failure testing and controlled operational trials before any real-world claim is made.